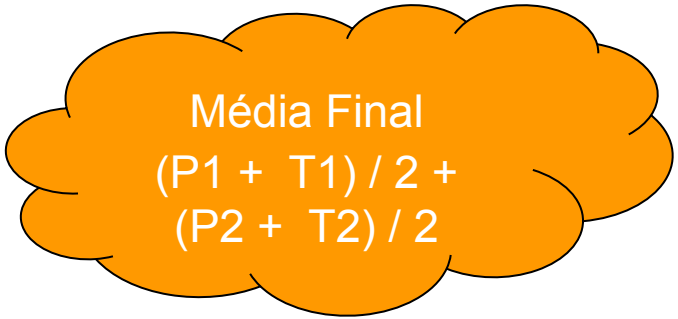
Gerência de configuração

Introdução e git

Prof. Diego Cardoso Borda Castro

Critérios de avaliação

- Provas individuais e sem consulta
  - Prova 1: 8 pontos
  - Prova 2: 8 pontos
  - Trabalhos: 2 pontos
  - Prova final e reposição (Matéria Toda)
  - Só será permitido repor uma prova

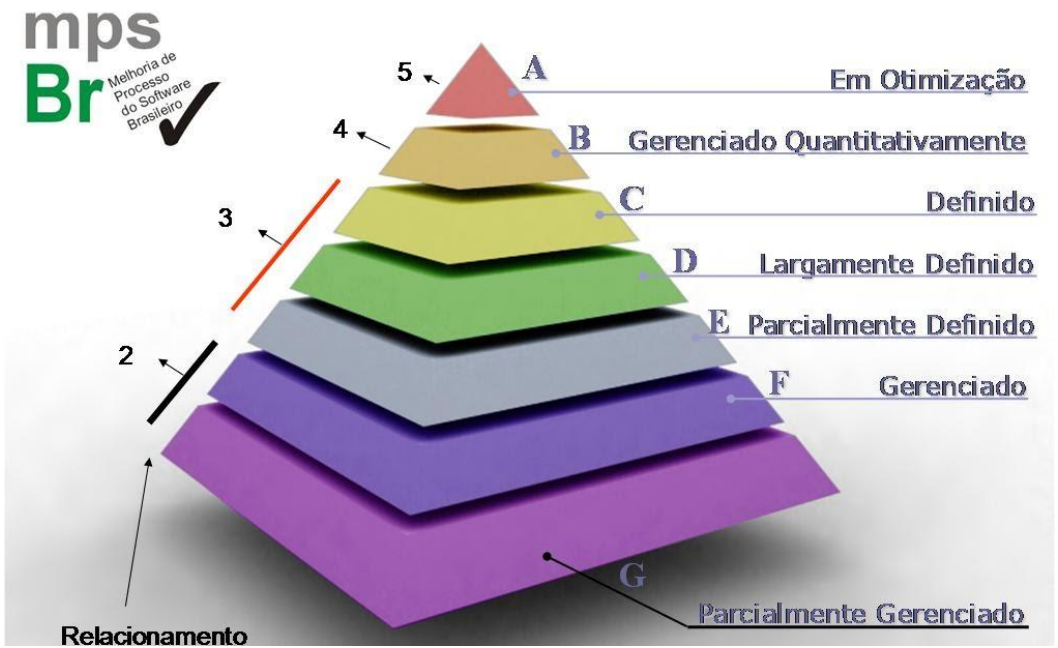


Modelo de maturidade

- Framework teórico
  - Mais voltado a definição de processos
  - Estabelece boas práticas
  - Aumenta previsibilidade, qualidade e governança
- Níveis evolutivos de capacidade organizacional
  - CMMI (Capability Maturity Model Integration)
    - 5 Níveis
  - MPS.BR (Melhoria de Processo do Software Brasileiro)
    - 7 Níveis

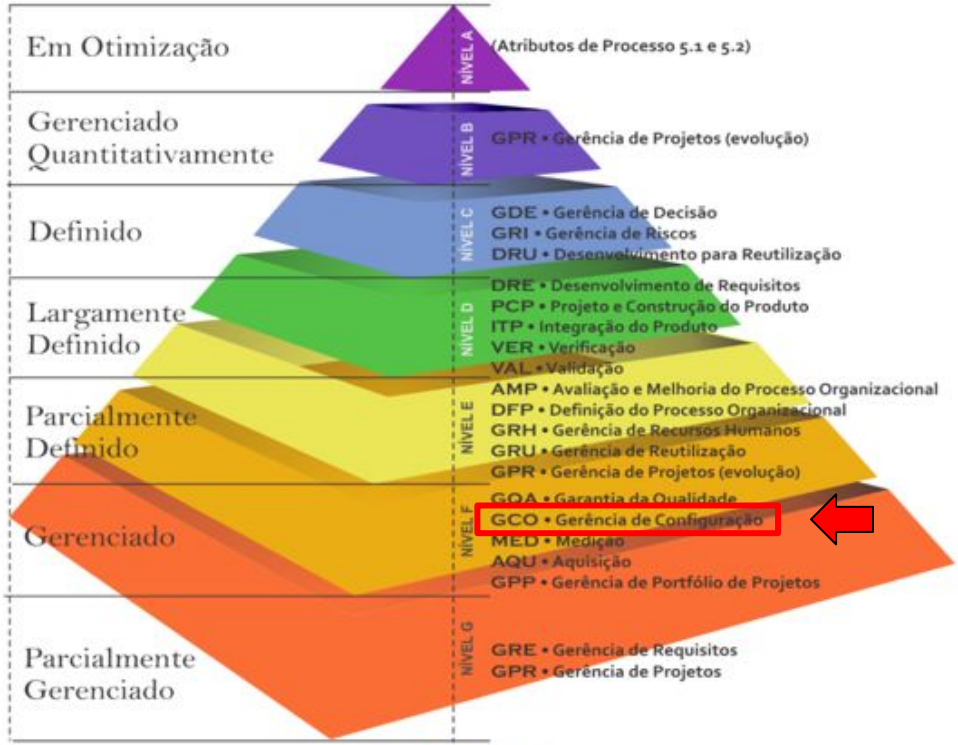
Modelo de maturidade

- Empresas brasileiras com dificuldades de efetivar o CMMI
- Medir Software
- Medir a empresa



Modelo de maturidade

- Identificar os Itens de Configuração
- Controlar Mudanças
- Controlar Versões
- Realizar Auditorias de Configuração
- Manter Registros de Status



DevOps

- CMMI
  - Define sugestões do que deve ser controlado e verifica se é feito
- Devops
  - Define como automatizar e operacionalizar esse controle na prática



DevOps

- Processo de desenvolvimento de software Tradicional
  - Planejamento
  - Desenvolvimento
    - Dev
      - Inovação
  - Testes
  - Implantação
    - Ops
      - Estabilidade



- Times diferentes
- Pouca comunicação
- Conhecimentos diferentes
- Propósitos diferentes

DevOps

- Cultura DevOps
  - Equipe multidisciplinar
  - Colaborativa
  - Foco na cultura e nas pessoas
  - Ferramentas



DevOps

- A cultura DevOps pe foca em 3 pilares
  - Fluxo
    - Tudo deve ser mapeável
    - Limitar o Work in progress
    - Trabalhar com pequenas entregas
      - Tempo de execução
      - Tempo de processamento
    - Reduzir número de transferência
    - Eliminar gargalos

DevOps

- A cultura DevOps pe foca em 3 pilares
  - Feedback
    - Utilizar recursos de observabilidade para verificar o fluxo das atividades, utilizar de métricas
      - Quantos testes passaram
    - Identificar problemas no início

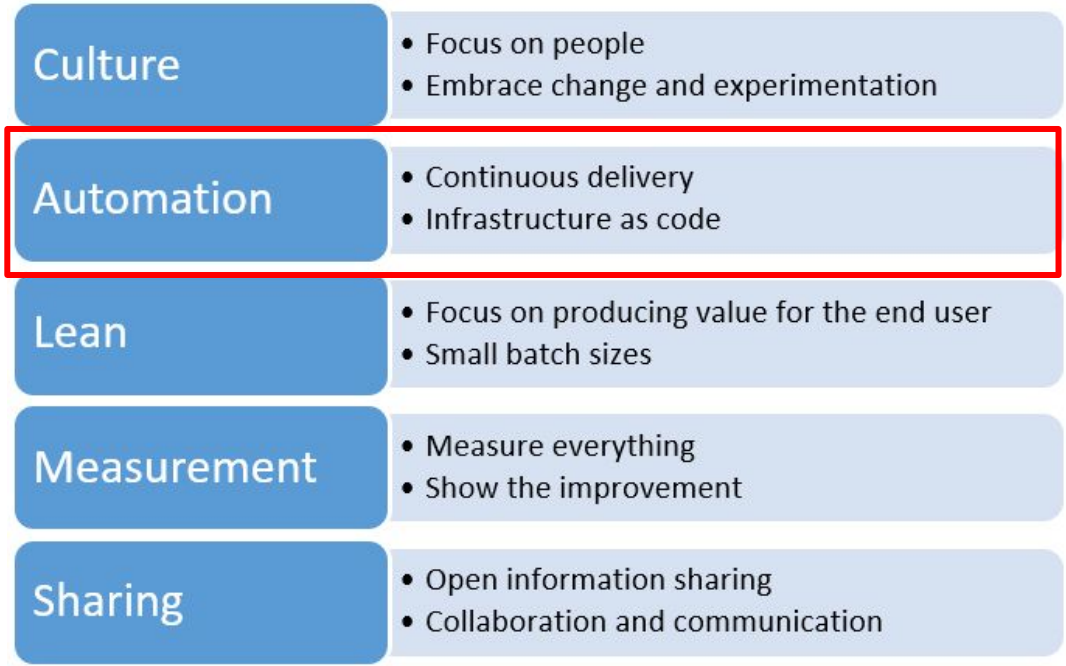
DevOps

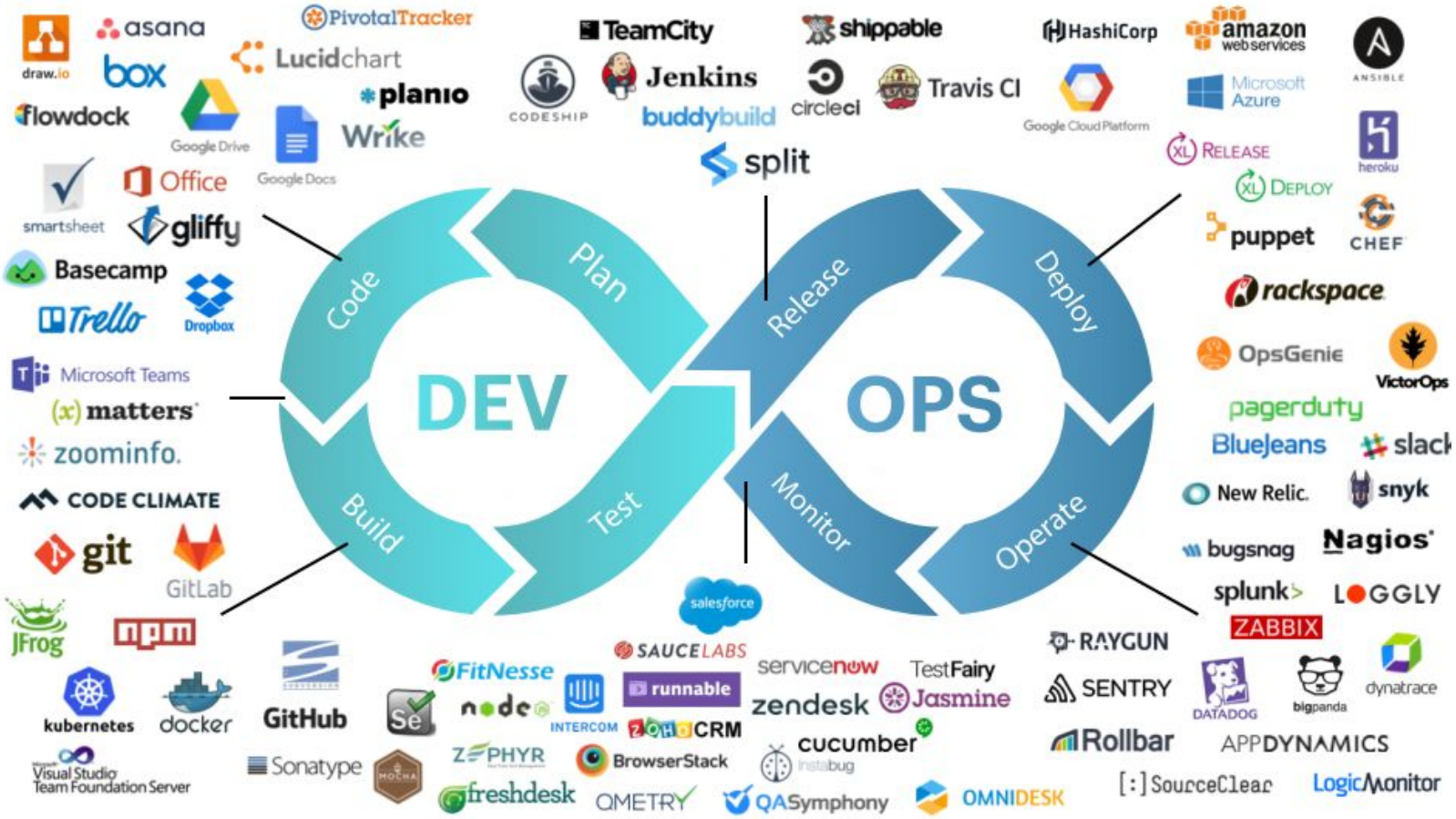
- A cultura DevOps pe foca em 3 pilares
  - Aprendizado e experimentação
    - Sem medo e sem culpa
      - Aprender com os erros
      - Não quero saber quem é o culpado
        - Aprender com ele
        - Resolver o problema
    - Difundir conhecimento



DevOps

- Framework CALMS
- DevOps é uma cultura
- Iremos focar nas ferramentas




Code
Plan
Release
Deploy
DEV
OPS
Build
Test
Monitor
Operate
draw.io
asana
PivotalTracker
TeamCity
shippable
HashiCorp
amazon web services
box
Lucidchart
Jenkins
Microsoft Azure
planio
CODESHIP
buddybuild
circleci
Travis CI
Google Cloud Platform
flowdock
Google Drive
Google Docs
Wrike
split
RELEASE
heroku
DEPLOY
Office
smartsheet
gliffy
puppet
CHEF
Basecamp
Trello
Dropbox
rackspace
OpsGenie
VictorOps
Microsoft Teams
(x) matters
pagerduty
BlueJeans
slack
zoominfo.
New Relic.
snyk
CODE CLIMATE
git
GitLab
bugsnag
Nagios
splunk>
LOGGLY
JFrog
npm
ZABBIX
RAYGUN
SAUCELABS
servicenow
TestFairy
FitNesse
runnable
zendesk
Jasmine
SENTRY
DATADOG
bigpanda
dynatrace
kubernetes
docker
GitHub
node
INTERCOM
ZOHO CRM
cucumber
Rollbar
APPDYNAMICS
Visual Studio Team Foundation Server
Sonatype
ZEPHYR
BrowserStack
freshdesk
QMETRY
QASymphony
OMNIDESK
[:]SourceClear
LogicMonitor
salesforce

DevOps

- Cronograma (P1):
  - Aula 1: Introdução + git inicial
  - Aula 2: Aprimorando conhecimento em Git
  - Aula 3: Introdução ao Docker
  - Aula 4: Aprimorando conhecimento em Docker
  - Aula 5: Introdução ao Github actions
  - Aula 6: Aprimorando conhecimento em Github actions + jenkins
  - Aula 7: Revisão pra prova
  - Prova

DevOps

- Cronograma (P2):
  - Aula 8: Introdução ao CI/CD com Kubernetes
  - Aula 9: CI/CD com Kubernetes
  - Aula 10: Introdução ao Terraform
  - Aula 11: Aprimorando conhecimento em Terraform
  - Aula 12: Introdução ao Ansible
  - Aula 13: Aprimorando conhecimento em Ansible
  - Aula 14: Revisão para prova
  - Prova

Gerenciamento de código

- Tudo é código
  - Nossa aplicação é código
  - Nossos testes são código
  - Nossa infraestrutura é código
- Teremos histórico de nossos projeto
  - Linha do tempo
- Auditoria



Gerenciamento de código

- Versão Centralizada (SVN)
- Versão Distribuído
  - Branch
    - Linha paralela de desenvolvimento dentro do seu projeto



Gerenciamento de código

- Git
  - Lançado em 2005 por Linus Torvalds para gerenciar o código fonte do Linux
  - Open source
  - Modelo Distribuído
  - Principal ferramenta hoje do mercado
  - Instalando o git
    - https://git-scm.com/install/windows
    - https://git-scm.com/install/linux



Gerenciamento de código

- Git config
  - Demonstra as informações atuais do seu git

```
git config --list
alias.co=checkout
alias.br=branch
alias.ci=commit
alias.st=status
alias.ac=git config --list add . && git commit -m
pull.rebase=true
user.name=Diego
user.email=diego.castro@cefet-rj.br
```

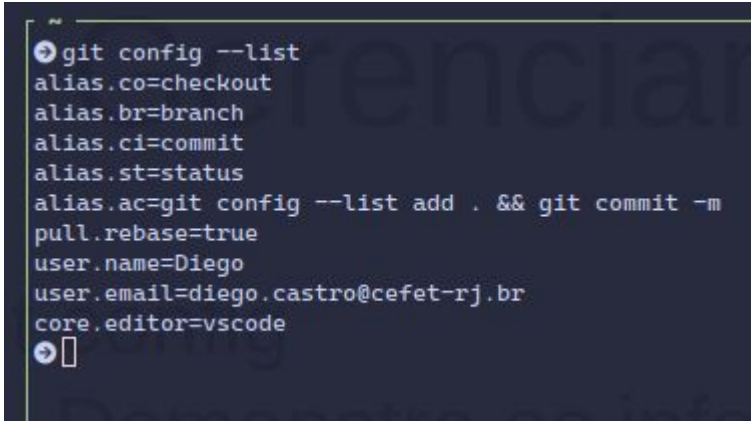
Alias: nome curto para um comando
git co (git checkout)

git config --global alias.ac "!git add . && git commit -m"

git ac "mensagem"

Gerenciamento de código

- Git config
  - Demonstra as informações atuais do seu git



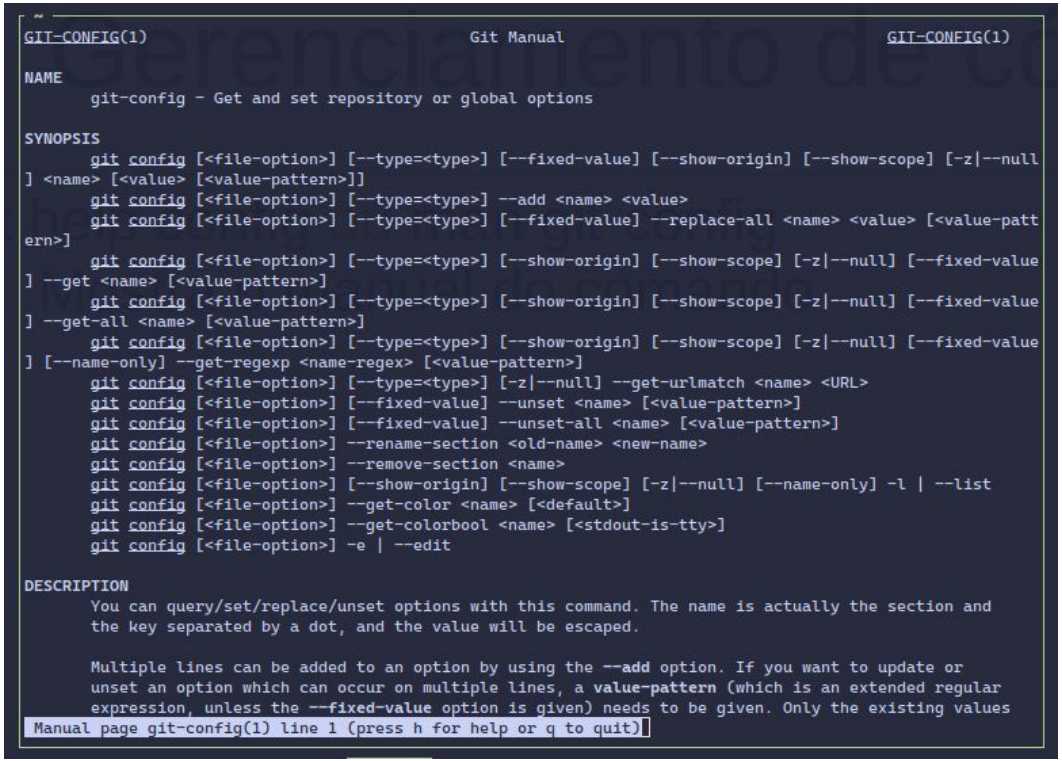
git config --global user.name “Diego”

git config --global user.email “diego.castro@cefet-rj.br”

git config --system core.editor vscode

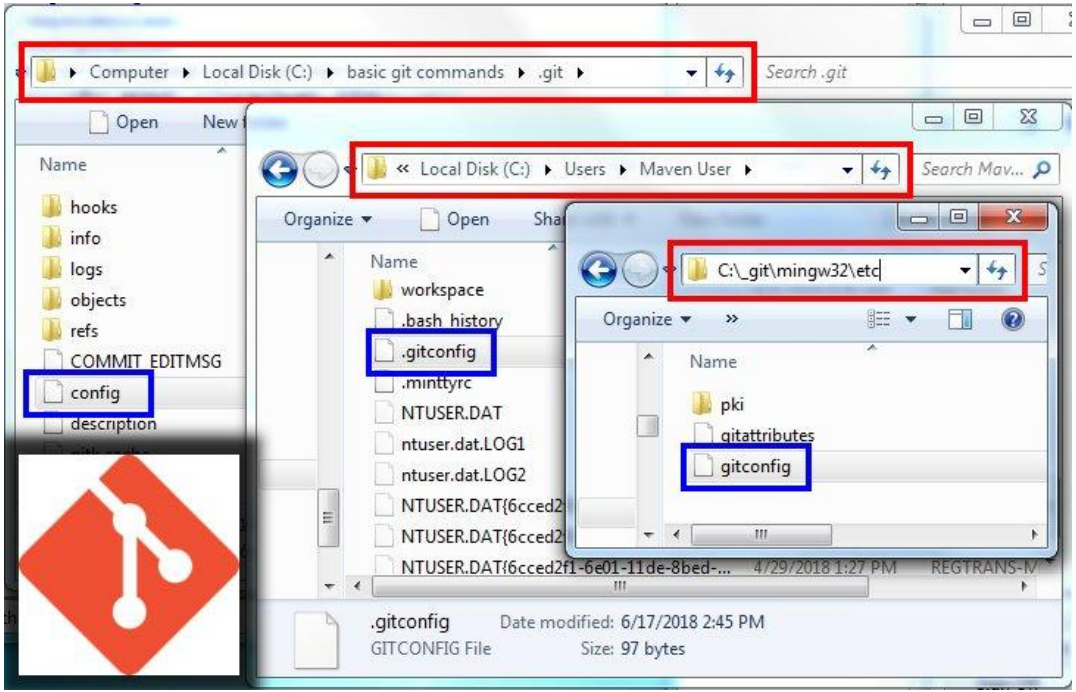
Gerenciamento de código

- git help config ou man git-config
  - Mostra o manual do comando



Gerenciamento de código

- --system
- --global
- --project



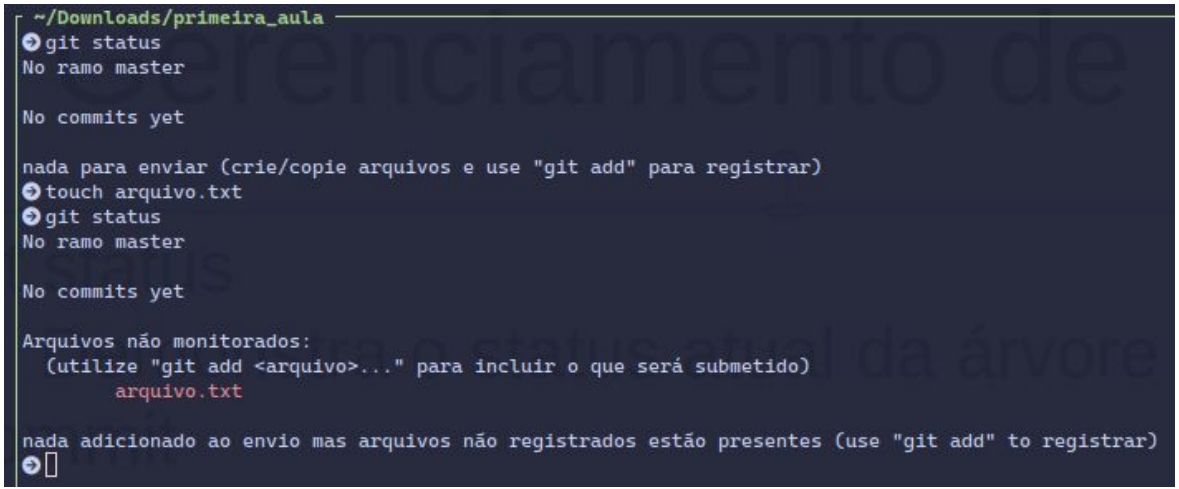
Gerenciamento de código

- git init
  - Inicializa um projeto git para versionar o seu projeto
  - Repositório
    - Diretório onde vou colocar todos os arquivos referentes a minha aplicação
    - Local
    - Remoto

```
git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Repositório vazio Git inicializado em  /home/diego/Downloads/primeira_aula/.git/
```

Gerenciamento de código

- Git status
  - Demonstra o status atual da árvore git



Gerenciamento de código

- git add .
- git add nome_arquivo
  - Adiciona o arquivo na área de stage
- git reset
  - Limpa a área de stage

```
git add arquivo.txt
git status
No ramo master

No commits yet

Mudanças a serem submetidas:
  (utilize "git rm --cached <arquivo>..." para não apresentar)
        new file:   arquivo.txt

git reset
git status
No ramo master

No commits yet

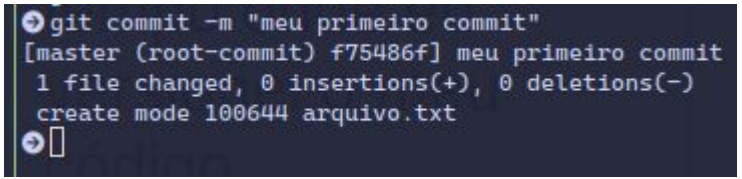
Arquivos não monitorados:
  (utilize "git add <arquivo>..." para incluir o que será submetido)
        arquivo.txt

nada adicionado ao envio mas arquivos não registrados estão presentes (use "git add" to registrar)
```

Gerenciamento de código

- git commit
  - Um ponto da linha do tempo onde eu versionei o meu código
- git commit -m “meu primeiro commit”
- git commit -a -m “meu primeiro commit”





Gerenciamento de código

- git log
  - Demonstra o log do repositório do git
- git log -p nome_arquivo
  - Demonstra o diff entre os commits

```
~/Downloads/primeira_aula
git log
commit f75486fbc1759ba4ac90ce8f9a49bfec867f5919 (HEAD -> master)
Author: Diego <diego.castro@cefet-rj.br>
Date:   Tue Feb 24 13:59:45 2026 -0300

    meu primeiro commit
```

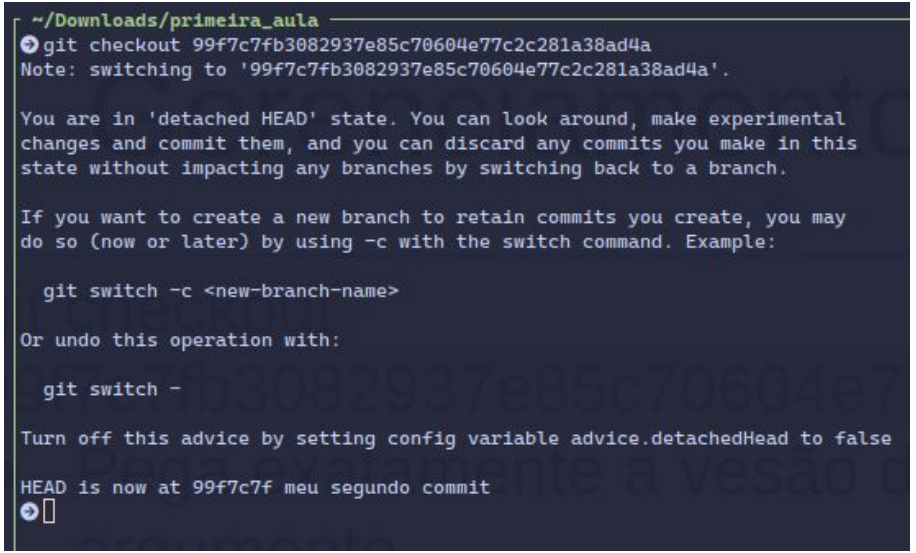
```
~/Downloads/primeira_aula
echo "teste" > arquivo.txt
git log -p arquivo.txt
commit 99f7c7fb3082937e85c70604e77c2c281a38ad4a (HEAD -> master)
Author: Diego <diego.castro@cefet-rj.br>
Date:   Tue Feb 24 14:07:09 2026 -0300

    meu segundo commit

diff --git a/arquivo.txt b/arquivo.txt
index e69de29..6f16acb 100644
--- a/arquivo.txt
+++ b/arquivo.txt
@@ -0,0 +1 @@
+teste
```

Gerenciamento de código

- git checkout 99f7c7fb3082937e85c70604e77c2c281a38ad4a
  - Pega exatamente a vesão do git que foi passada por argumento



Gerenciamento de código

- git reset
  - Move o ponteiro da branch para outro commit
  - Head é commit atual
  - --soft HEAD~1

Comando	Move o HEAD?	Mantém o Stage (Índice)?	Mantém os Arquivos (Working Dir)?
--soft	✅ Sim	✅ Sim	✅ Sim
--mixed (padrão)	✅ Sim	❌ Não	✅ Sim
--hard	✅ Sim	❌ Não	❌ Não

Gerenciamento de código

```
~/Downloads/primeira_aula
echo "teste 2" > arquivo.txt
git commit -a -m "meu terceiro commit"
[detached HEAD 2358333] meu terceiro commit
 1 file changed, 1 insertion(+), 1 deletion(-)
git status
HEAD detached from 99f7c7f
nothing to commit, working tree clean
git log
commit 2358333bba535621df543adfd92ab3d9afcaf342 (HEAD)
Author: Diego <diego.castro@cefet-rj.br>
Date:   Tue Feb 24 14:20:16 2026 -0300

    meu terceiro commit

commit 99f7c7fb3082937e85c70604e77c2c281a38ad4a (master)
Author: Diego <diego.castro@cefet-rj.br>
Date:   Tue Feb 24 14:07:09 2026 -0300

    meu segundo commit

commit f75486fbc1759ba4ac90ce8f9a49bfec867f5919
Author: Diego <diego.castro@cefet-rj.br>
Date:   Tue Feb 24 13:59:45 2026 -0300

    meu primeiro commit
```

```
git reset --soft HEAD~1
git status
HEAD detached at 99f7c7f
Mudanças a serem submetidas:
  (use "git restore --staged <file>..." to unstage)
        modified:   arquivo.txt

git log
commit 99f7c7fb3082937e85c70604e77c2c281a38ad4a (HEAD, master)
Author: Diego <diego.castro@cefet-rj.br>
Date:   Tue Feb 24 14:07:09 2026 -0300

    meu segundo commit

commit f75486fbc1759ba4ac90ce8f9a49bfec867f5919
Author: Diego <diego.castro@cefet-rj.br>
Date:   Tue Feb 24 13:59:45 2026 -0300

    meu primeiro commit
```

Gerenciamento de código

- git ignore
  - Arquivos que não devem subir para o git
    - Binários, compilados, logs, pasta de dependências, etc
  - Arquivos que não devem ser rastreáveis
  - Criar arquivo .gitignore
    - node_modules/ [Pasta]
    - *.log [Qualquer arquivo]
    - config.json [específico]
    - !bin/*.log [Adiciona]
    - **/build/ [Qualquer diretório]
    - teste [qualquer arquivo e pata em qualquer diretório]

Gerenciamento de código

- Branch
  - Facilitar o trabalho colaborativo
  - Criar ramificações do projeto
  - Tipos
    - main
    - develop
    - feature/nome-da-feature
    - bugfix/nome-do-bug
    - hotfix/nome-do-hotfix
    - release/nome-da-release

Gerenciamento de código

- git branch feature/teste
- git branch
- git branch -m feature/teste
- git branch -d feature/teste
- git branch -D feature/teste



Gerenciamento de código

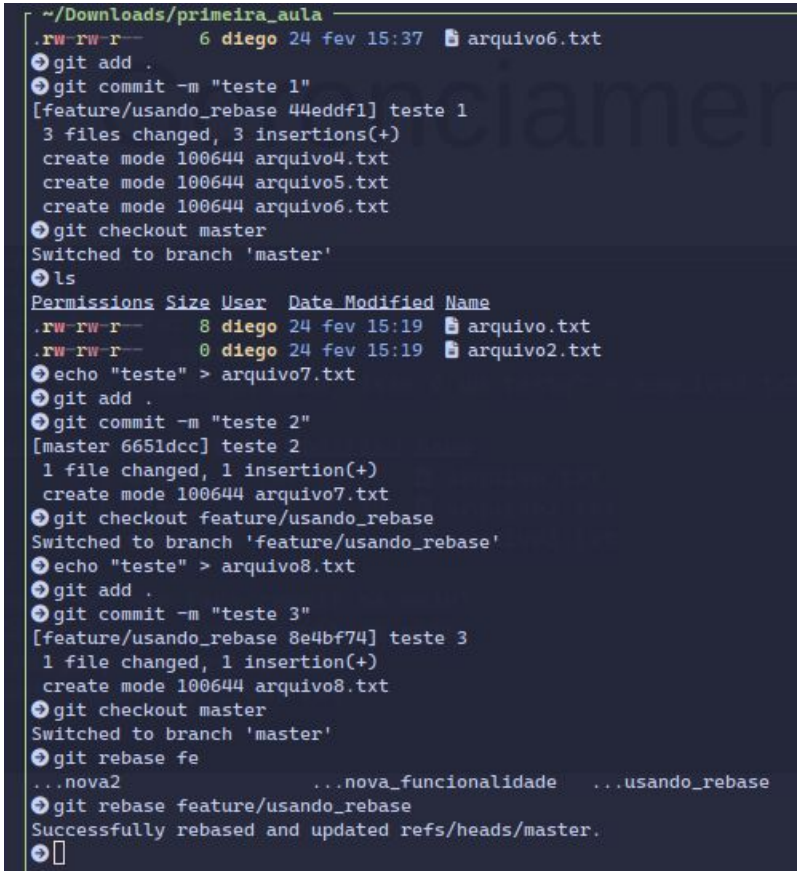
- Git merge
  - Combina duas branchs
  - git merge nome-da-branch -m "mensagem do merge"

```
~/Downloads/primeira_aula
git branch
* feature/nova_funcionalidade
  master
git checkout -b feature/nova2
Switched to a new branch 'feature/nova2'
touch arquivo2.txt
git commit -a -m "meu quarto commit"
[feature/nova2 35602cd] meu quarto commit
 1 file changed, 1 insertion(+), 1 deletion(-)
git status
```

```
~/Downloads/primeira_aula
git merge fe
...2                    ..._funcionalidade
git merge feature/nova_funcionalidade
Already up to date.
git merge feature/nova2
Updating 99f7c7f..abf697e
Fast-forward
 arquivo.txt  | 2 +-
 arquivo2.txt | 0
 2 files changed, 1 insertion(+), 1 deletion(-)
 create mode 100644 arquivo2.txt
```

Gerenciamento de código

- Git rebase X git merge
  - O merge sempre faz um commit a mais
  - O rebase vai pegar os commits da branch e mover para a outra que eu quero



Gerenciamento de código

- Git rebase X git merge
  - O merge sempre faz um commit a mais
  - O rebase vai pegar os commits da branch e mover para a outra que eu quero

```
~/Downloads/primeira_aula
commit 6d30de2cb867c829db6dee57b31186a1df879c9e (HEAD -> master)
Author: Diego <diego.castro@cefet-rj.br>
Date:   Tue Feb 24 15:39:31 2026 -0300

    teste 2

commit de23f3e03ebed7dd98baec03481c67889ac3effa
Author: Diego <diego.castro@cefet-rj.br>
Date:   Tue Feb 24 15:34:59 2026 -0300

    deletando arquivo3

commit 8e4bf749e8f20c76c3ef5281209f3ea257bd3688 (feature/usando_rebase)
Author: Diego <diego.castro@cefet-rj.br>
Date:   Tue Feb 24 15:40:42 2026 -0300

    teste 3

commit 44eddf171f5ba136676a1d49fb610c3bedec8f3e
Author: Diego <diego.castro@cefet-rj.br>
Date:   Tue Feb 24 15:38:43 2026 -0300

    teste 1

commit 98c1641c74e035b7ba7755c77f457b9490b65cca
Author: Diego <diego.castro@cefet-rj.br>
Date:   Tue Feb 24 15:30:29 2026 -0300

    teste

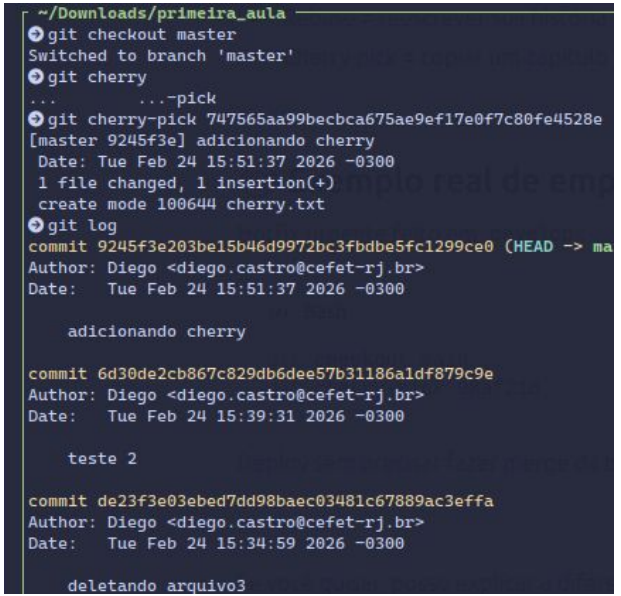
commit b780c839624c0e1f137aab6901103d90f84178c6
Author: Diego <diego.castro@cefet-rj.br>
Date:   Tue Feb 24 15:24:44 2026 -0300

:
```

Gerenciamento de código

- Git merge
  - Pega um commit específico e joga na branch atual





Gerenciamento de código

- Git tag
  - Uma nova versão do projeto
  - Representa um marco para o projeto

```
~/Downloads/primeira_aula
git status
No ramo master
nothing to commit, working tree clean
git tag
git tag -a v2.0 -m "versão 2.0"
git tag
v2.0
```

```
~/Downloads/primeira_aula
git tag -a v.0 -m "versão 1.0" 44eddf171f5ba136676a1d49fb610c3bedec8f3e
git tag
v.0
v2.0
git show v.0
tag v.0
Tagger: Diego <diego.castro@cefet-rj.br>
Date:   Tue Feb 24 16:00:33 2026 -0300

versão 1.0

commit 44eddf171f5ba136676a1d49fb610c3bedec8f3e (tag: v.0)
Author: Diego <diego.castro@cefet-rj.br>
Date:   Tue Feb 24 15:38:43 2026 -0300

    teste 1

diff --git a/arquivo4.txt b/arquivo4.txt
new file mode 100644
index 0000000..6f16acb
--- /dev/null
+++ b/arquivo4.txt
@@ -0,0 +1 @@
+teste
diff --git a/arquivo5.txt b/arquivo5.txt
new file mode 100644
index 0000000..6f16acb
--- /dev/null
+++ b/arquivo5.txt
@@ -0,0 +1 @@
+teste
diff --git a/arquivo6.txt b/arquivo6.txt
```